

INSTITUTO FEDERAL DE MINAS GERAIS (IFMG)
CAMPUS BAMBUÍ
BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

Disciplina: BiSuEEA. 512 – Sistemas Embarcados
Professor: Williams L. Nicomedes

Henrique Augusto G. Fernandes

LISTA DE EXERCÍCIOS Nº3 – CIRCUITOS COMBINACIONAIS

SUMÁRIO

1	INTRODUÇÃO	3
2	DESENVOLVIMENTO	4
2.1	Análise da Função Booleana	4
2.2	Tabela-Verdade	4
2.3	Item 1: Diagrama Esquemático	5
2.3.1	<i>Componentes Utilizados</i>	5
2.3.2	<i>Circuito Implementado</i>	5
2.3.3	<i>Simulação do Circuito Esquemático</i>	5
2.4	Item 2: Código VHDL	6
2.4.1	<i>Código Desenvolvido</i>	6
2.4.2	<i>Estrutura do Código</i>	6
2.4.3	<i>Geração do Bloco Lógico</i>	7
2.5	Comparação dos Resultados	7
2.5.1	<i>Análise das Saídas</i>	8
3	CONCLUSÃO	9

1 INTRODUÇÃO

Este documento apresenta a resolução da Lista de Exercícios nº3 da disciplina de Sistemas Embarcados, referente ao 2º semestre de 2025. O objetivo é implementar um circuito combinacional que realize a função booleana dada, utilizando tanto o diagrama esquemático com portas lógicas quanto o código em VHDL, e comparar os resultados obtidos em ambas as abordagens.

A função booleana a ser implementada é:

$$z = A \cdot B + \overline{(C + D)} \quad (1.1)$$

O desenvolvimento foi realizado no software Quartus II, utilizando arquivos de projeto do tipo *Block Diagram/Schematic File* e *VHDL File*, além de arquivos de verificação do tipo *University Program VWF* para simulação das formas de onda.

2 DESENVOLVIMENTO

2.1 Análise da Função Booleana

A função booleana $z = A \cdot B + \overline{(C + D)}$ pode ser decomposta nas seguintes operações lógicas:

- operação AND entre as entradas A e B, resultando em $x = A \cdot B$;
- operação OR entre as entradas C e D, resultando em $y = C + D$;
- operação NOT sobre o resultado anterior, resultando em $w = \bar{y} = \overline{(C + D)}$;
- operação OR entre os resultados das etapas (a) e (c), resultando em $z = x + w$.

2.2 Tabela-Verdade

A Tabela 1 apresenta todas as 16 combinações possíveis para as 4 entradas (A, B, C, D) e o respectivo valor da saída z.

Tabela 1 – Tabela-verdade da função $z = A \cdot B + \overline{(C + D)}$

D	C	B	A	C+D	$\overline{(C + D)}$	A·B	z
0	0	0	0	0	1	0	1
0	0	0	1	0	1	0	1
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	1
0	1	0	0	1	0	0	0
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	0
0	1	1	1	1	0	1	1
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	0
1	0	1	0	1	0	0	0
1	0	1	1	1	0	1	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	0
1	1	1	0	1	0	0	0
1	1	1	1	1	0	1	1

Fonte: Elaborado pelo autor, 2025.

2.3 Item 1: Diagrama Esquemático

O primeiro item da lista solicita a construção do diagrama esquemático que implementa a função booleana utilizando portas lógicas no Quartus II.

2.3.1 Componentes Utilizados

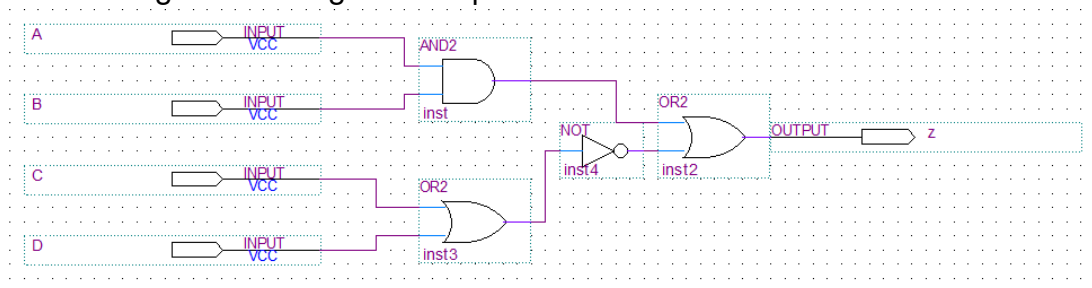
Para implementar a função $z = A \cdot B + \overline{(C + D)}$, foram utilizados os seguintes componentes:

- uma porta AND de 2 entradas (AND2) – para realizar $A \cdot B$;
- uma porta OR de 2 entradas (OR2) – para realizar $C + D$;
- uma porta NOT – para realizar $\overline{(C + D)}$;
- uma porta OR de 2 entradas (OR2) – para combinar os resultados finais.

2.3.2 Circuito Implementado

A Figura 1 apresenta o diagrama esquemático construído no Quartus II, mostrando as conexões entre as portas lógicas para implementar a função booleana.

Figura 1 – Diagrama esquemático do circuito combinacional



Fonte: Elaborado pelo autor, 2025.

2.3.3 Simulação do Circuito Esquemático

Para verificar o funcionamento do circuito, foi criado um arquivo de formas de onda (University Program VWF) com todas as combinações possíveis das entradas A, B, C e D. Os sinais de entrada foram configurados com os seguintes períodos para gerar um padrão de contagem binária:

- entrada A: período de $1 \mu\text{s}$ (alterna mais rápido);
- entrada B: período de $2 \mu\text{s}$;
- entrada C: período de $4 \mu\text{s}$;
- entrada D: período de $8 \mu\text{s}$ (alterna mais lento).

2.4 Item 2: Código VHDL

O segundo item solicita a construção de um código em VHDL que implemente a mesma função booleana e a comparação das saídas com o item anterior.

2.4.1 Código Desenvolvido

O Quadro 1 apresenta o código VHDL desenvolvido para implementar o bloco lógico BlocoComb4.

Quadro 1 – Código VHDL do bloco combinacional BlocoComb4

```
library ieee;
use ieee. std_logic_1164.all;

-- Entidade: Define as entradas e saídas do bloco
entity BlocoComb4 is
port(
A, B, C, D : in std_logic;
z : out std_logic
);
end BlocoComb4;

-- Arquitetura: Define a lógica interna do circuito
architecture Funcionamento of BlocoComb4 is
signal x, y, w : std_logic;

begin
-- x = A AND B
x <= A AND B;

-- y = C OR D
y <= C OR D;

-- w = NOT(C OR D)
w <= NOT(y);

-- z = (A AND B) OR NOT(C OR D)
z <= x OR w;

end Funcionamento;
```

Fonte: Elaborado pelo autor, 2025.

2.4.2 Estrutura do Código

O código VHDL está estruturado da seguinte forma:

- a) **Bibliotecas:** inclusão da biblioteca `ieee` e do pacote `std_logic_1164`

que define o tipo `std_logic`;

- b) **Entidade:** declaração das 4 entradas (A, B, C, D) e 1 saída (z), todas do tipo `std_logic`;
- c) **Arquitetura:** definição da lógica interna utilizando 3 sinais intermediários:
 - x: armazena o resultado de $A \cdot B$;
 - y: armazena o resultado de $C + D$;
 - w: armazena o resultado de $\overline{(C + D)}$.

2.4.3 Geração do Bloco Lógico

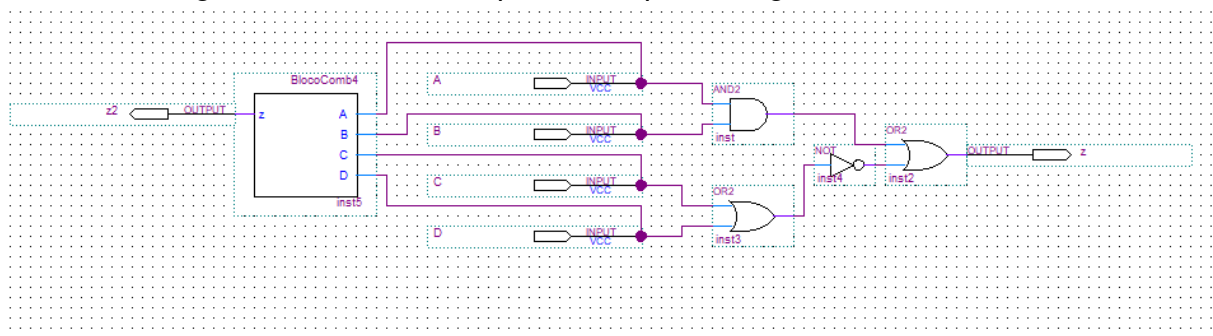
Após a compilação do código VHDL, foi gerado o símbolo do bloco lógico através dos comandos:

- a) **Processing** → **Analyze Current File**;
- b) **File** → **Create/Update** → **Create Symbol Files for Current File**.

2.5 Comparação dos Resultados

A Figura 2 apresenta o esquemático completo contendo tanto o circuito implementado com portas lógicas discretas quanto o bloco VHDL, permitindo a comparação direta das saídas.

Figura 2 – Circuito completo com portas lógicas e bloco VHDL

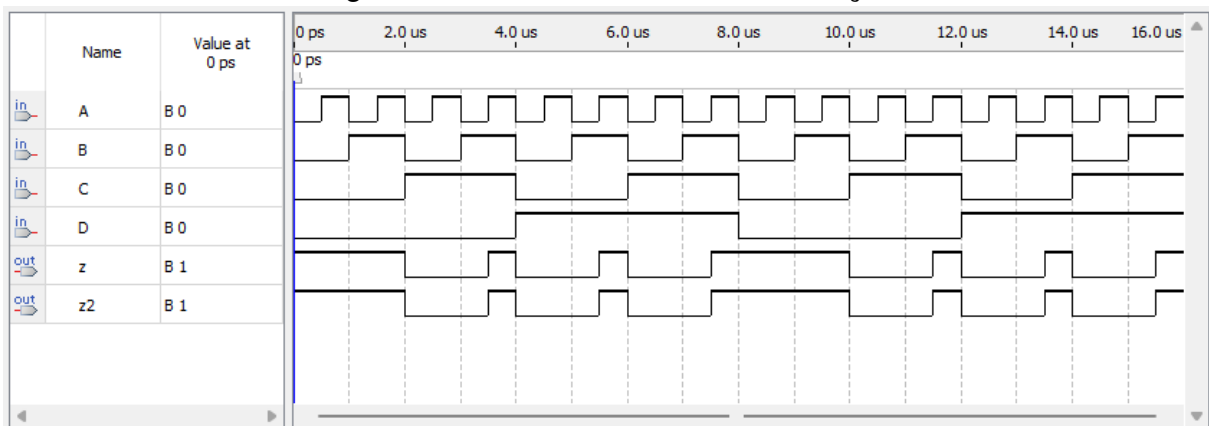


Fonte: Elaborado pelo autor, 2025.

A Figura 3 apresenta as formas de onda resultantes da simulação, onde:

- a) as entradas A, B, C e D são mostradas como sinais de entrada;
- b) a saída **z** corresponde ao circuito implementado com portas lógicas discretas;
- c) a saída **z2** corresponde ao bloco VHDL (BlocoComb4).

Figura 3 – Formas de onda da simulação



Fonte: Elaborado pelo autor, 2025.

2.5.1 Análise das Saídas

Observando as formas de onda na Figura 3, pode-se verificar que:

- as saídas z e z2 são **idênticas** em todos os instantes de tempo;
- quando $C = 0$ e $D = 0$, a saída é sempre 1 (devido ao termo $\overline{(C + D)} = 1$);
- quando $C = 1$ ou $D = 1$ (ou ambos), a saída depende exclusivamente do termo $A \cdot B$;
- a saída é 1 apenas quando $A = B = 1$ e $(C = 1$ ou $D = 1)$, ou quando $C = D = 0$.

Os resultados confirmam que ambas as implementações (diagrama esquemático e código VHDL) produzem comportamentos funcionalmente equivalentes, validando a corretude do projeto.

3 CONCLUSÃO

Este trabalho apresentou a implementação da função booleana $z = A \cdot B + \overline{(C + D)}$ utilizando duas abordagens distintas no Quartus II:

- a) diagrama esquemático com portas lógicas (AND, OR, NOT);
- b) código em linguagem VHDL.

A simulação através de formas de onda demonstrou que ambas as implementações produzem resultados idênticos para todas as 16 combinações possíveis das entradas, confirmando a equivalência funcional entre o circuito discreto e a descrição em hardware através do código VHDL.

Este exercício ilustra a metodologia de projeto de sistemas digitais, onde diferentes níveis de abstração (esquemático e HDL) podem ser utilizados para descrever o mesmo comportamento lógico. A abordagem modular, utilizando blocos lógicos criados em VHDL e posteriormente integrados em esquemáticos maiores, demonstra o conceito de “dividir para conquistar” no desenvolvimento de sistemas embarcados.